

Final Project

Digital Systems Lab 2025

Group 6

Aditya Tripathy - EE24BTECH11001

Agamjot Singh - EE24BTECH11002

Abhimanyu Koushik Garimella - EE24BTECH11024

May 7, 2025

1 Problem Statement 1 - Digital Clock

1. Time Display:

- Display time in the format hh:mm:ss
- Support for both 12-hour (AM/PM) and 24-hour modes
- AM/PM indicator for 12-hour format

2. Timer Functionality:

- Countdown timer up to 10 minutes
- Buzzer when timer reaches zero

3. Date Display:

- Support for date in the format dd-mm-yyyy (limited from Jan 2020 to April 2025)
- Resets to Jan 1, 2020

4. Alarm Functionality:

- Ability to set an alarm (time-based), which activates a second buzzer when triggered

2 Architecture and Modularization

The code is modularized into 5 submodules and a main driver module

1. **Clock Handler:** Increments time and manages rollover in clock
2. **Display Handler:** Manages time display and switching between 12 hour and 24 hour formats
3. **Date Handler:** Handles date functionality and calendar functionality
4. **Timer Handler:** Implements countdown timer functionality
5. **Alarm Handler:** Manages alarm settings and triggering
6. **Main Driver:** Integrates all modules into a single main module which drives the entire clock

3 Module Implementations

3.1 Clock Handler Module

The clock handler module increments the current time.

```
module clock_handler (  
    input clk,  
    input reset,  
    input set_time,  
    input [7:0] input_sec,  
    input [7:0] input_min,  
    input [7:0] input_hour,  
  
    output reg [7:0] current_24_sec,  
    output reg [7:0] current_24_min,  
    output reg [7:0] current_24_hour  
);
```

1. **Time Tracking:** The module increments seconds, minutes, and hours based on the clock signal, implementing proper rollover logic.
2. **Time Setting:** When `set_time` is active, the module updates the current time with user-provided values.

```
// synchronous time set and update  
always @(posedge clk or posedge reset) begin  
    if (reset) begin  
        current_24_sec <= 0;  
        current_24_min <= 0;  
        current_24_hour <= 0;  
    end else if (set_time == 1'd1) begin  
        current_24_sec <= input_sec;  
        current_24_min <= input_min;  
        current_24_hour <= input_hour;  
    end else begin  
        if (current_24_sec == 8'd59) begin  
            current_24_sec <= 8'd0;  
            if (current_24_min == 8'd59) begin  
                current_24_min <= 8'd0;  
                if (current_24_hour == 8'd23) begin  
                    current_24_hour <= 8'd0;  
                end else begin  
                    current_24_hour <= current_24_hour + 1;  
                end  
            end else begin  
                current_24_min <= current_24_min + 1;  
            end  
        end else begin  
            current_24_sec <= current_24_sec + 1;  
        end  
    end  
end  
end
```

The time tracking logic is implemented in a synchronous block that responds to the clock signal and reset. When neither reset nor `set_time` is active, the module increments the time values, handling rollovers from seconds to minutes to hours appropriately.

3.2 Display Handler Module

The display handler module displays the current time in 12/24 hour format or when the timer is running, it displays the time left.

```

module display_handler (
    input clk,
    input hour_format, // 1 for 12 hour, 0 for 24 hour format
    input timer_running,

    input [7:0] current_24_sec,
    input [7:0] current_24_min,
    input [7:0] current_24_hour,

    input [7:0] timer_min,
    input [7:0] timer_sec,

    output reg [7:0] display_sec,
    output reg [7:0] display_min,
    output reg [7:0] display_hour,
    output reg is_pm
);

```

1. **Format Conversion:** The module converts between 24-hour and 12-hour formats based on the hour_format input:

```

display_sec <= current_24_sec;
display_min <= current_24_min;

if (hour_format == 1'b1) begin // 12-hour format
    if (current_24_hour == 0) begin
        display_hour <= 8'd12; // 12 AM (midnight)
    end else if (current_24_hour == 12) begin
        display_hour <= 8'd12; // 12 PM (noon)
    end else if (current_24_hour > 12) begin
        display_hour <= current_24_hour - 12; // PM hours
    end else begin
        display_hour <= current_24_hour; // AM hours
    end
end else begin // 24-hour format
    display_hour <= current_24_hour;
end

if (hour_format == 1'b1) begin // 12-hour format
    is_pm <= (current_24_hour >= 12) ? 1'b1 : 1'b0;
end else begin
    is_pm <= 1'b0; // Don't care in 24-hour mode
end

```

2. **Timer Duration:** The module switches the display from the clock to the timer duration when the timer is running (the clock still runs in the background).

```

if(timer_running) begin
    display_sec <= timer_sec;
    display_min <= timer_min;
    display_hour <= 8'd0;
end

```

3.3 Date Handler Module

The date handler module manages the calendar functionality, tracking days, months, and years with proper handling of month lengths and leap years.

```

module date_handler (
    input clk,
    input reset,

```

```

input set_date,
input [7:0] input_day,
input [7:0] input_month,
input [15:0] input_year,

// Clock time inputs to detect day changes
input [7:0] current_24_hour,
input [7:0] current_24_min,
input [7:0] current_24_sec,

// Date outputs
output reg [ 7:0] current_day,
output reg [ 7:0] current_month,
output reg [15:0] current_year
);

// Internal Registers
reg [7:0] days_in_current_month;

```

1. **Date Tracking:** The module advances the date at midnight (23:59:59) by monitoring the current time.
2. **Month Length Calculation:** The module determines the number of days in each month, including leap year calculations for February:

```

// First determine days in current month without using a function
case (current_month)
  8'd1, 8'd3, 8'd5, 8'd7, 8'd8, 8'd10, 8'd12:
    days_in_current_month = 8'd31; // Jan, Mar, May, Jul, Aug, Oct,
    Dec
  8'd4, 8'd6, 8'd9, 8'd11: days_in_current_month = 8'd30; // Apr, Jun,
    Sep, Nov
  8'd2: begin // February
    // Check leap year
    if (((current_year % 4 == 0) && (current_year % 100 != 0)) || (
      current_year % 400 == 0))
      days_in_current_month = 8'd29; // Leap year
    else days_in_current_month = 8'd28; // Non-leap year
    end
  default: days_in_current_month = 8'd30; // Default case
endcase

```

3. **Date Advancement:** The module handles month and year transitions when the day advances past the end of a month:

```

// Update date
if (current_day == days_in_current_month) begin
  current_day <= 8'd1; // Reset day to 1

  if (current_month == 8'd12) begin
    current_month <= 8'd1; // Reset month to January
    current_year <= current_year + 16'd1; // Increment year
  end else begin
    current_month <= current_month + 8'd1; // Increment month
  end
end else begin
  current_day <= current_day + 8'd1; // Increment day
end

```

The date handler properly accounts for leap years in February by implementing the standard leap year calculation algorithm. It also handles the varying number of days in different months.

3.4 Timer Handler Module

The timer handler module implements a countdown timer with a maximum duration of 10 minutes.

```
module timer_handler (
    input clk,
    input reset,
    input start_timer,
    input stop_timer,
    input set_timer,
    input [7:0] input_min,
    input [7:0] input_sec,

    output reg [7:0] timer_min,
    output reg [7:0] timer_sec,
    output reg timer_running,
    output reg timer_buzzer
);

// Internal Registers
reg [7:0] max_min = 8'd10; // maximum minutes = 10
```

Key features of the timer handler:

1. **Timer Setting:** The module allows setting a timer duration with a maximum of 10 minutes.
2. **Timer Control:** The module provides start and stop functionality through control signals.

```
if (reset) begin
    timer_min <= 8'd0;
    timer_sec <= 8'd0;
    timer_running <= 1'b0;
    timer_buzzer <= 1'b0;
end else if (set_timer) begin
    if (input_min > max_min) begin
        // Limit to maximum 10 minutes
        timer_min <= max_min;
        timer_sec <= input_sec;
    end else begin
        timer_min <= input_min;
        timer_sec <= input_sec;
    end
    timer_running <= 1'b0;
    timer_buzzer <= 1'b0;
end else if (start_timer) begin
    // Start the timer
    timer_running <= 1'b1;
    timer_buzzer <= 1'b0;
end else if (stop_timer) begin
    // Stop the timer
    timer_running <= 1'b0;
end
```

3. **Countdown Logic:** When running, the timer decrements seconds and minutes appropriately:

```
if (timer_running) begin
    // Timer is running, decrement
    if (timer_sec == 8'd0) begin
        if (timer_min == 8'd0) begin
            // Timer has reached zero
            timer_running <= 1'b0;
            timer_buzzer <= 1'b1; // Buzzer on
        end else begin
            // Decrement minute, reset seconds to 59
            timer_min <= timer_min - 8'd1;
        end
    end
end
```

```

        timer_sec <= 8'd59;
    end
end else begin
    // Decrement seconds
    timer_sec <= timer_sec - 8'd1;
end
end
end

```

4. **Buzzer Activation:** The module activates a buzzer signal when the timer reaches zero.

The timer handler includes a safety check to ensure that the timer cannot be set for more than 10 minutes, as specified in the requirements. It also provides a clean interface for starting, stopping, and setting the timer.

3.5 Alarm Handler Module

The alarm handler module implements all the alarm functionality including Snooze and Stop functionality.

```

module alarm_handler (
    input clk,
    input set_alarm,

    input [7:0] current_24_sec,
    input [7:0] current_24_min,
    input [7:0] current_24_hour,

    input [7:0] alarm_input_sec,
    input [7:0] alarm_input_min,
    input [7:0] alarm_input_hour,

    input snooze_alarm,
    input stop_alarm,

    output reg alarm_buzzer
);

// Internal Registers
reg [7:0] input_sec_internal;
reg [7:0] input_min_internal;
reg [7:0] input_hour_internal;

```

1. **Alarm Setting:** The module stores the alarm time when the `set_alarm` signal is active.

```

if(set_alarm) begin
    input_sec_internal <= input_sec;
    input_min_internal <= input_min;
    input_hour_internal <= input_hour;
    alarm_buzzer <= 1;
end

```

2. **Alarm Triggering:** The module compares the current time with the set alarm time and activates the buzzer when they match:
3. **Snooze and Stop Mechanism:** The alarm keeps on buzzing until snoozed or stopped. Snooze has a delay of 5 clock cycles.

```

always @(posedge clk) begin
    // Stop alarm functionality
    if(stop_alarm) begin
        is_buzzing <= 0;
        alarm_buzzer <= 0;
    end
end

```

```

        counter <= 0;
    end
    // Snooze alarm functionality
    else if(snooze_alarm) begin
        is_snoozed <= 1;
        alarm_buzzer <= 0;
        is_buzzing <= 0;
        counter <= 0;
    end
    // Set alarm functionality
    else if(set_alarm) begin
        input_sec_internal <= alarm_input_sec;
        input_min_internal <= alarm_input_min;
        input_hour_internal <= alarm_input_hour;
        is_snoozed <= 0;
        alarm_buzzer <= 0;
        is_buzzing <= 0;
        counter <= 0;
    end
    // Snooze timer functionality
    else if(is_snoozed) begin
        alarm_buzzer <= 0;
        is_buzzing <= 0;
        counter <= counter + 1;
        if(counter >= 5) begin // After snooze period (300 clock cycles)
            is_snoozed <= 0;
            alarm_buzzer <= 1;
            is_buzzing <= 1;
            counter <= 0;
        end
    end
    // Alarm matching current time
    else if ((input_sec_internal == current_24_sec &&
        input_min_internal == current_24_min &&
        input_hour_internal == current_24_hour) || is_buzzing) begin
        if(!is_buzzing) is_buzzing <= 1;
        alarm_buzzer <= 1;
    end
    // Default state - alarm not active
    else begin
        alarm_buzzer <= 0;
    end
end
end

```

The alarm handler uses internal registers to store the current time when the alarm is set. It then compares this stored time with the alarm time to determine when to activate the buzzer.

3.6 Main Driver Module

The main driver module integrates all the individual modules into a complete digital clock system.

```

module main_driver (
    input        clk,
    input        reset,
    input        hour_format,
    input        set_time,
    input        set_date,
    input        set_alarm,
    input        set_timer,
    input        start_timer,
    input        stop_timer,

    input [7:0] input_sec,

```

```

input  [7:0] input_min,
input  [7:0] input_hour,
input  [7:0] input_day,
input  [7:0] input_month,
input  [15:0] input_year,
input  [7:0] timer_input_min,
input  [7:0] timer_input_sec,
input  [7:0] alarm_time_sec,
input  [7:0] alarm_time_min,
input  [7:0] alarm_time_hour,

// Various outputs from all modules
...
);

```

The main driver instantiates each of the four functional modules and connects them appropriately:

```

// Instantiate the clock_handler module
clock_handler clock_module (
    .clk(clk),
    .reset(reset),
    .hour_format(hour_format),
    .set_time(set_time),
    .input_sec(input_sec),
    .input_min(input_min),
    .input_hour(input_hour),
    .current_24_sec(current_24_sec),
    .current_24_min(current_24_min),
    .current_24_hour(current_24_hour),
    .display_sec(display_sec),
    .display_min(display_min),
    .display_hour(display_hour),
    .is_pm(is_pm)
);

// Instantiate the display_handler module
display_handler display_module (
    // connections...
);

// Instantiate the date_handler module
date_handler date_module (
    // connections...
);

// Instantiate the timer_handler module
timer_handler timer_module (
    // connections...
);

// Instantiate the alarm_handler module
alarm_handler alarm_module (
    // connections...
);

```

The main driver serves as the top-level module that drives all the individual functional modules together.

4 Testbench and Verification

A comprehensive testbench was developed to verify the functionality of the digital clock system. The testbench includes tests for:

1. Setting the time and verifying time advancement
2. Setting the date and verifying date changes at midnight
3. Setting and triggering alarms
4. Setting, starting, stopping, and completing timers
5. Switching between 12-hour and 24-hour formats

The testbench uses a simulated clock with a period of 1 second and monitors all relevant signals to verify correct operation.

4.1 Testbench 1

```
'timescale 1s/1ms

module digital_clock_tb;
    // Clock and reset signals
    reg clk;
    reg reset;

    // Clock time control signals
    reg set_time;
    reg [7:0] input_sec;
    reg [7:0] input_min;
    reg [7:0] input_hour;
    reg hour_format;

    // Date control signals
    reg set_date;
    reg [7:0] input_day;
    reg [7:0] input_month;
    reg [15:0] input_year;

    // Timer control signals
    reg set_timer;
    reg start_timer;
    reg stop_timer;
    reg [7:0] timer_input_min;
    reg [7:0] timer_input_sec;

    // Alarm control signals
    reg set_alarm;
    reg [7:0] alarm_time_sec;
    reg [7:0] alarm_time_min;
    reg [7:0] alarm_time_hour;

    // ... (outputs and instantiation)

initial begin
    // Initialize all inputs
    reset = 1;
    // ... (initialization)

    // Apply reset for 20s
    #20 reset = 0;

    // Test 1: Set the time to 23:59:50
    #10;
    set_time = 1;
    input_sec = 50;
    input_min = 59;
```

```

input_hour = 23;
#10;
set_time = 0;

// Test 2: Set the date to December 31, 2025
#10;
set_date = 1;
input_day = 31;
input_month = 12;
input_year = 2025;
#10; set_date = 0;

// ... (additional tests)

initial begin
    $monitor("Time: %3d | Clock: %02d:%02d:%02d | Display: %02d:%02d:%02d %c%c
    | Date: %02d/%02d/%4d\nTimer: %02d:%02d (Running: %b, Done: %b) | Alarm:
    %b\n...",
        $time,
        current_24_hour, current_24_min, current_24_sec,
        display_hours, display_minutes, display_seconds,
        am_pm_indicator[15:8], am_pm_indicator[7:0],
        current_day, current_month, current_year,
        timer_min, timer_sec, timer_running, timer_buzzer,
        alarm_buzzer,
        // ... (additional monitored signals)
    );
end

```

4.1.1 Timing Diagram

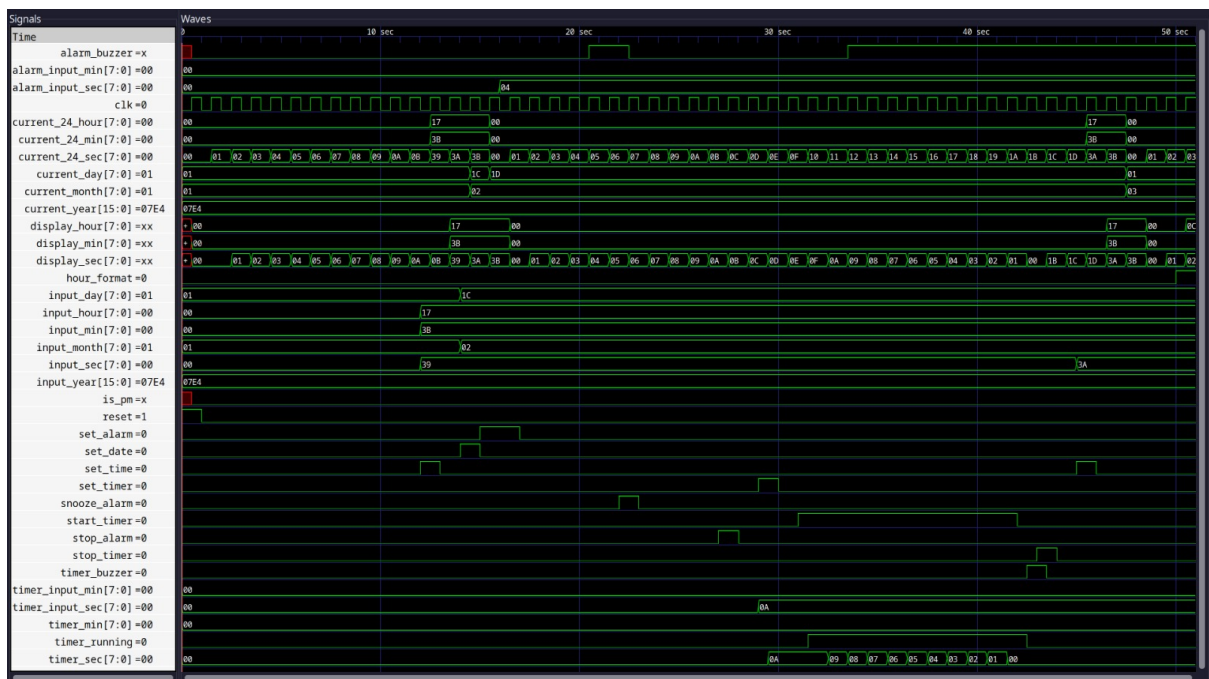


Figure 1: Timing diagram for Testbench 1

4.1.2 Terminal Output

This is too long to be shown in this report, but for testing thoroughly it was required.

4.2 Testbench 2

```
'timescale 1s/1ms

module clock_tb;
    // Inputs
    reg clk;
    reg reset;
    reg hour_format;
    reg set_time;
    reg set_date;
    reg set_alarm;
    reg snooze_alarm;
    reg stop_alarm;
    reg set_timer;
    reg start_timer;
    reg stop_timer;
    // ... ( outputs and instantiation of DUT )

    // AM/PM indicator logic
    reg is_pm;
    reg [15:0] am_pm_indicator;

    // Display logic for AM/PM indicator
    // ONLY FOR DISPLAY PURPOSE -- HAS NO EFFECT ON THE WORKING
    always @(*) begin
        is_pm = (current_24_hour >= 8'd12);

        if (hour_format) begin
            if (is_pm)
                am_pm_indicator = {"P", "M"}; // 'P' = 8'h50, 'M' = 8'h4D
            else
                am_pm_indicator = {"A", "M"}; // 'A' = 8'h41, 'M' = 8'h4D
        end else begin
            am_pm_indicator = "  "; // Two spaces if 24-hour
        end
    end

    // Clock generation (1 Hz clock for seconds)
    initial begin
        clk = 0;
        forever #0.5 clk = ~clk; // 1 second period
    end

    // Test sequence
    initial begin
        // Initialize all inputs
        reset = 0;
        // ... ( initialization to default )

        // First reset
        #1 reset = 1;
        #2 reset = 0;
        $display("Reset done at %0ds", $time);

        // Set date to Dec 31 2022, time to 23:53:55
        input_day = 8'd31;
        input_month = 8'd12;
        input_year = 16'd2022;
        #1 set_date = 1;
        #1 set_date = 0;

        input_hour = 8'd23;
```

```

input_min = 8'd59;
input_sec = 8'd58;
#1 set_time = 1;
$display("Set date to Dec 31 2022, time to 23:59:58 at %0ds", $time);
#1 set_time = 0;

$display("Waiting for next day to start...");
#2;
$display("Next day started at %0ds", $time);

// Change to AM-PM mode
#1 hour_format = 1;
$display("Changed to AM-PM mode at %0ds", $time);

// Wait for 10 seconds
#10;

// Change date to Feb 28 2020 and time to 23:53:55
input_day = 8'd28;
input_month = 8'd2;
input_year = 16'd2020;
#1 set_date = 1;
#1 set_date = 0;

input_hour = 8'd23;
input_min = 8'd59;
input_sec = 8'd57;
#1 set_time = 1;
$display("Set date to Feb 28 2020, time to 23:59:57 at %0ds", $time);
#1 set_time = 0;

$display("Waiting for next day to start...");
#4;
$display("Next day started at %0ds", $time);
// Wait 10 seconds
hour_format = 0;
$display("Change to 24-Hour format");
#10;

// Set an alarm for 00:00:30
$display("Set alarm for 00:00:30 at %0ds", $time);
alarm_input_hour = 8'd0;
alarm_input_min = 8'd0;
alarm_input_sec = 8'd30;
#1 set_alarm = 1;
#1 set_alarm = 0;

// Wait for alarm to trigger (should be in about 20 seconds)
wait(alarm_buzzer);
$display("Alarm triggered at %0ds", $time);

// Snooze
#5 snooze_alarm = 1;
#2 snooze_alarm = 0;
$display("Alarm snoozed at %0ds", $time);

// Wait 5 seconds and stop alarm
#5;
#2 stop_alarm = 1;
$display("Alarm stopped at %0ds", $time);
#2 stop_alarm = 0;

// Set a timer for 1 minute (60 seconds)

```

```

    timer_input_min = 8'd0;
    timer_input_sec = 8'd2;
    #1 set_timer = 1;
    #1 set_timer = 0;
    #1 start_timer = 1;
    #1 start_timer = 0;
    $display("Set timer for 2 seconds at %0ds", $time);

    #5 stop_timer = 1;
    // End simulation
    #1 $finish;
end

// Monitor the clock outputs
initial begin
    $monitor("Time: %0ds, Date: %0d/%0d/%0d, Time: %0d:%0d:%0d %s, Alarm: %b, Timer: %b",
        $time,
        current_day, current_month, current_year,
        display_hour, display_min, display_sec,
        am_pm_indicator,
        alarm_buzzer,
        timer_buzzer);
end

initial begin
    $dumpfile("wave.vcd");
    $dumpvars(0, clock_tb);
end
endmodule

```

4.2.1 Timing diagram

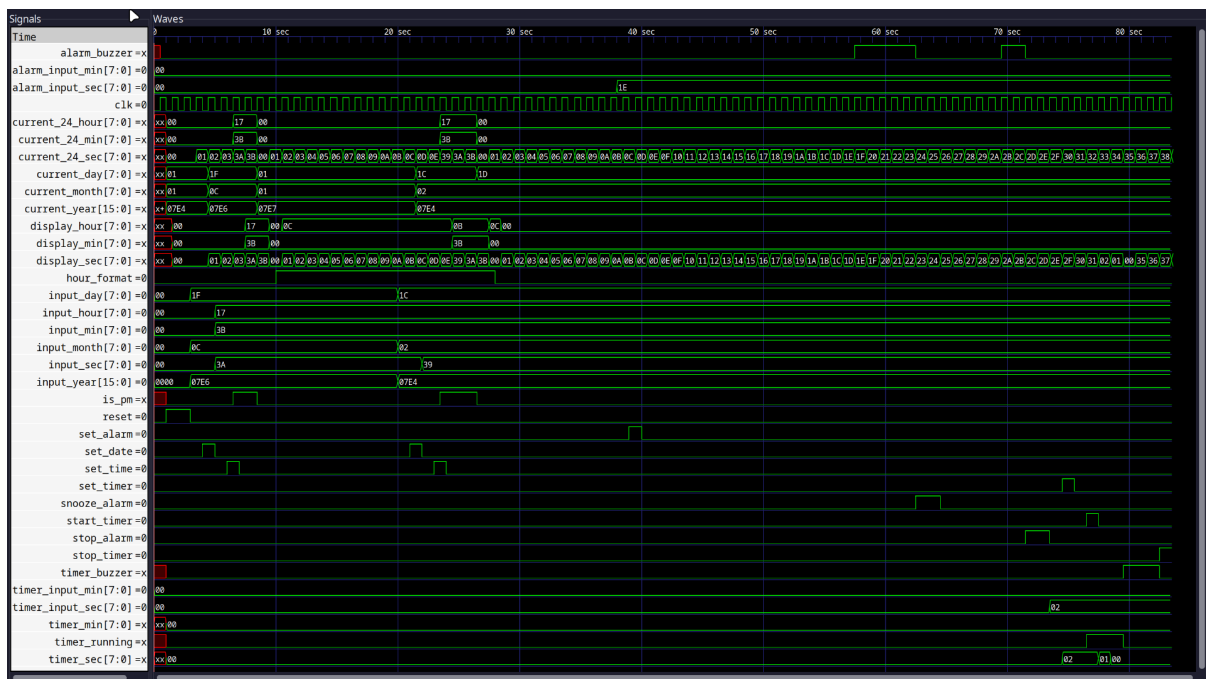


Figure 2: Timing Diagram for TestBench 2

4.2.2 Terminal Output

```

Time: 0s, Date: x/x/x, Time: x:x:x , Alarm: x, Timer: x
Time: 1s, Date: x/x/x, Time: x:x:x , Alarm: 0, Timer: x
Time: 1s, Date: 1/1/2020, Time: x:x:x , Alarm: 0, Timer: 0
Time: 2s, Date: 1/1/2020, Time: 0:0:0 , Alarm: 0, Timer: 0
Reset done at 3s
Time: 5s, Date: 31/12/2022, Time: 0:0:1 , Alarm: 0, Timer: 0
Time: 6s, Date: 31/12/2022, Time: 0:0:2 , Alarm: 0, Timer: 0
Set date to Dec 31 2022, time to 23:59:58 at 6s
Time: 7s, Date: 31/12/2022, Time: 0:0:3 , Alarm: 0, Timer: 0
Waiting for next day to start...
Time: 8s, Date: 31/12/2022, Time: 23:59:58 , Alarm: 0, Timer: 0
Time: 9s, Date: 1/1/2023, Time: 23:59:59 , Alarm: 0, Timer: 0
Next day started at 9s
Time: 10s, Date: 1/1/2023, Time: 0:0:0 , Alarm: 0, Timer: 0
Changed to AM-PM mode at 10s
Time: 10s, Date: 1/1/2023, Time: 0:0:0 AM, Alarm: 0, Timer: 0
Time: 11s, Date: 1/1/2023, Time: 12:0:1 AM, Alarm: 0, Timer: 0
Time: 12s, Date: 1/1/2023, Time: 12:0:2 AM, Alarm: 0, Timer: 0
Time: 13s, Date: 1/1/2023, Time: 12:0:3 AM, Alarm: 0, Timer: 0
Time: 14s, Date: 1/1/2023, Time: 12:0:4 AM, Alarm: 0, Timer: 0
Time: 15s, Date: 1/1/2023, Time: 12:0:5 AM, Alarm: 0, Timer: 0
Time: 16s, Date: 1/1/2023, Time: 12:0:6 AM, Alarm: 0, Timer: 0
Time: 17s, Date: 1/1/2023, Time: 12:0:7 AM, Alarm: 0, Timer: 0
Time: 18s, Date: 1/1/2023, Time: 12:0:8 AM, Alarm: 0, Timer: 0
Time: 19s, Date: 1/1/2023, Time: 12:0:9 AM, Alarm: 0, Timer: 0
Time: 20s, Date: 1/1/2023, Time: 12:0:10 AM, Alarm: 0, Timer: 0
Time: 21s, Date: 1/1/2023, Time: 12:0:11 AM, Alarm: 0, Timer: 0
Time: 22s, Date: 28/2/2020, Time: 12:0:12 AM, Alarm: 0, Timer: 0
Time: 23s, Date: 28/2/2020, Time: 12:0:13 AM, Alarm: 0, Timer: 0
Set date to Feb 28 2020, time to 23:59:57 at 23s
Time: 24s, Date: 28/2/2020, Time: 12:0:14 PM, Alarm: 0, Timer: 0
Waiting for next day to start...
Time: 25s, Date: 28/2/2020, Time: 11:59:57 PM, Alarm: 0, Timer: 0
Time: 26s, Date: 28/2/2020, Time: 11:59:58 PM, Alarm: 0, Timer: 0
Time: 27s, Date: 29/2/2020, Time: 11:59:59 AM, Alarm: 0, Timer: 0
Time: 28s, Date: 29/2/2020, Time: 12:0:0 AM, Alarm: 0, Timer: 0
Next day started at 28s
Change to 24-Hour format
Time: 28s, Date: 29/2/2020, Time: 12:0:0 , Alarm: 0, Timer: 0
Time: 29s, Date: 29/2/2020, Time: 0:0:1 , Alarm: 0, Timer: 0
Time: 30s, Date: 29/2/2020, Time: 0:0:2 , Alarm: 0, Timer: 0
Time: 31s, Date: 29/2/2020, Time: 0:0:3 , Alarm: 0, Timer: 0
Time: 32s, Date: 29/2/2020, Time: 0:0:4 , Alarm: 0, Timer: 0
Time: 33s, Date: 29/2/2020, Time: 0:0:5 , Alarm: 0, Timer: 0
Time: 34s, Date: 29/2/2020, Time: 0:0:6 , Alarm: 0, Timer: 0
Time: 35s, Date: 29/2/2020, Time: 0:0:7 , Alarm: 0, Timer: 0
Time: 36s, Date: 29/2/2020, Time: 0:0:8 , Alarm: 0, Timer: 0
Time: 37s, Date: 29/2/2020, Time: 0:0:9 , Alarm: 0, Timer: 0
Time: 38s, Date: 29/2/2020, Time: 0:0:10 , Alarm: 0, Timer: 0
Set alarm for 00:00:30 at 38s
Time: 39s, Date: 29/2/2020, Time: 0:0:11 , Alarm: 0, Timer: 0
Time: 40s, Date: 29/2/2020, Time: 0:0:12 , Alarm: 0, Timer: 0
Time: 41s, Date: 29/2/2020, Time: 0:0:13 , Alarm: 0, Timer: 0
Time: 42s, Date: 29/2/2020, Time: 0:0:14 , Alarm: 0, Timer: 0
Time: 43s, Date: 29/2/2020, Time: 0:0:15 , Alarm: 0, Timer: 0
Time: 44s, Date: 29/2/2020, Time: 0:0:16 , Alarm: 0, Timer: 0
Time: 45s, Date: 29/2/2020, Time: 0:0:17 , Alarm: 0, Timer: 0
Time: 46s, Date: 29/2/2020, Time: 0:0:18 , Alarm: 0, Timer: 0
Time: 47s, Date: 29/2/2020, Time: 0:0:19 , Alarm: 0, Timer: 0
Time: 48s, Date: 29/2/2020, Time: 0:0:20 , Alarm: 0, Timer: 0
Time: 49s, Date: 29/2/2020, Time: 0:0:21 , Alarm: 0, Timer: 0
Time: 50s, Date: 29/2/2020, Time: 0:0:22 , Alarm: 0, Timer: 0
Time: 51s, Date: 29/2/2020, Time: 0:0:23 , Alarm: 0, Timer: 0

```

```

Time: 52s, Date: 29/2/2020, Time: 0:0:24, Alarm: 0, Timer: 0
Time: 53s, Date: 29/2/2020, Time: 0:0:25, Alarm: 0, Timer: 0
Time: 54s, Date: 29/2/2020, Time: 0:0:26, Alarm: 0, Timer: 0
Time: 55s, Date: 29/2/2020, Time: 0:0:27, Alarm: 0, Timer: 0
Time: 56s, Date: 29/2/2020, Time: 0:0:28, Alarm: 0, Timer: 0
Time: 57s, Date: 29/2/2020, Time: 0:0:29, Alarm: 0, Timer: 0
Alarm triggered at 58s
Time: 58s, Date: 29/2/2020, Time: 0:0:30, Alarm: 1, Timer: 0
Time: 59s, Date: 29/2/2020, Time: 0:0:31, Alarm: 1, Timer: 0
Time: 60s, Date: 29/2/2020, Time: 0:0:32, Alarm: 1, Timer: 0
Time: 61s, Date: 29/2/2020, Time: 0:0:33, Alarm: 1, Timer: 0
Time: 62s, Date: 29/2/2020, Time: 0:0:34, Alarm: 1, Timer: 0
Time: 63s, Date: 29/2/2020, Time: 0:0:35, Alarm: 0, Timer: 0
Time: 64s, Date: 29/2/2020, Time: 0:0:36, Alarm: 0, Timer: 0
Alarm snoozed at 65s
Time: 65s, Date: 29/2/2020, Time: 0:0:37, Alarm: 0, Timer: 0
Time: 66s, Date: 29/2/2020, Time: 0:0:38, Alarm: 0, Timer: 0
Time: 67s, Date: 29/2/2020, Time: 0:0:39, Alarm: 0, Timer: 0
Time: 68s, Date: 29/2/2020, Time: 0:0:40, Alarm: 0, Timer: 0
Time: 69s, Date: 29/2/2020, Time: 0:0:41, Alarm: 0, Timer: 0
Time: 70s, Date: 29/2/2020, Time: 0:0:42, Alarm: 1, Timer: 0
Time: 71s, Date: 29/2/2020, Time: 0:0:43, Alarm: 1, Timer: 0
Alarm stopped at 72s
Time: 72s, Date: 29/2/2020, Time: 0:0:44, Alarm: 0, Timer: 0
Time: 73s, Date: 29/2/2020, Time: 0:0:45, Alarm: 0, Timer: 0
Time: 74s, Date: 29/2/2020, Time: 0:0:46, Alarm: 0, Timer: 0
Time: 75s, Date: 29/2/2020, Time: 0:0:47, Alarm: 0, Timer: 0
Time: 76s, Date: 29/2/2020, Time: 0:0:48, Alarm: 0, Timer: 0
Time: 77s, Date: 29/2/2020, Time: 0:0:49, Alarm: 0, Timer: 0
Set timer for 2 seconds at 78s
Time: 78s, Date: 29/2/2020, Time: 0:0:2, Alarm: 0, Timer: 0
Time: 79s, Date: 29/2/2020, Time: 0:0:1, Alarm: 0, Timer: 0
Time: 80s, Date: 29/2/2020, Time: 0:0:0, Alarm: 0, Timer: 1
Time: 81s, Date: 29/2/2020, Time: 0:0:53, Alarm: 0, Timer: 1
Time: 82s, Date: 29/2/2020, Time: 0:0:54, Alarm: 0, Timer: 1
Time: 83s, Date: 29/2/2020, Time: 0:0:55, Alarm: 0, Timer: 0
Time: 84s, Date: 29/2/2020, Time: 0:0:56, Alarm: 0, Timer: 0

```

5 Overview of the Design

5.1 Clock Synchronization

One of the key challenges in the design was ensuring proper synchronization between the different modules. The clock handler drives the time advancement, which in turn triggers date changes at midnight. This required careful coordination between the modules to ensure that date changes occur precisely at the right moment.

5.2 Leap Year Calculation

Implementing the leap year logic in the date handler required careful consideration of the leap year rules. The standard rule is that a year is a leap year if it is divisible by 4, except for years divisible by 100 but not by 400. This logic was implemented in the date handler to correctly determine the number of days in February.

5.3 Format Conversion

Converting between 12-hour and 24-hour formats required special handling of edge cases such as midnight (00:00) and noon (12:00). The clock handler includes logic to handle these special cases correctly.

5.4 Timer and Alarm Functionality

The timer and alarm modules needed to be designed to work independently of the main clock functionality. The timer counts down from a set value, while the alarm triggers when the current time matches the set alarm time. Both modules needed to handle edge cases such as timer reaching zero and alarm time matching the current time.

6 Problem Statement 2 - T-Junction Traffic Light Controller

6.1 System Overview

The traffic light controller manages a T-junction with the following features:

- Two traffic signals (T1 and T2) with three states: Green (00), Yellow (01), and Red (10)
- Pedestrian walk signals for both directions (T1_WALK and T2_WALK)
- Warning buzzers for pedestrian crossings (buzzer1 and buzzer2)
- Emergency vehicle priority handling for both left and right approaches

6.2 State Machine Design

The controller implements a 6-state finite state machine:

- S1: T1 Green, T2 Red, T2_WALK enabled
- S2: T1 Yellow, T2 Red, T2_WALK enabled
- S3: T1 Red, T2 Red, both WALK signals enabled
- S4: T1 Red, T2 Red, both WALK signals enabled, buzzer2 active
- S5: T1 Red, T2 Green, T1_WALK enabled
- S6: T1 Red, T2 Yellow, T1_WALK enabled, buzzer1 active

6.3 Emergency Handling

The controller implements priority-based emergency handling:

- Emergency_Right has priority over Emergency_Left
- During Emergency_Right, both signals turn red
- During Emergency_Left, T1 is red and T2 is green
- Emergency duration is at least T_EM (9 seconds)
- If emergency signal remains active after T_EM, the emergency state continues

6.4 Finite State Machine (FSM) Design

The traffic light controller is implemented using a Moore-type Finite State Machine with six states:

State	T1	T2	T1_WALK	T2_WALK	Buzzer1	Buzzer2
S1	GREEN	RED	0	1	0	0
S2	YELLOW	RED	0	1	0	0
S3	RED	RED	1	1	0	0
S4	RED	RED	1	1	0	1
S5	RED	GREEN	1	0	0	0
S6	RED	YELLOW	1	0	1	0

Table 1: State Table for Normal Operation

6.5 State Transition Table

The state transitions follow a predetermined sequence in normal operation, with specific timing for each state:

7 Verilog Implementation

7.1 Signal Encoding

The traffic signals use the following 2-bit encoding:

- **00:** Green light
- **01:** Yellow light
- **10:** Red light

7.2 Module Interface

The module has the following input and output ports:

```
module traffic_light_controller(  
    input          clk,           // Clock input  
    input          reset,         // Reset signal  
    input          Emergency_Left, // Emergency signal from left  
    input          Emergency_Right, // Emergency signal from right  
    output reg [1:0] T1,          // Traffic light 1 (00:Green, 01:Yellow,  
                                10:Red)  
    output reg [1:0] T2,          // Traffic light 2 (00:Green, 01:Yellow,  
                                10:Red)  
    output reg      T1_WALK,      // Pedestrian signal for T1  
    output reg      T2_WALK,      // Pedestrian signal for T2  
    output reg      buzzer1,      // Audible indicator 1  
    output reg      buzzer2      // Audible indicator 2  
);
```

7.3 State Machine Design

7.3.1 State Definitions

The controller uses a 3-bit state register to represent six distinct operational states:

```
// State encoding  
localparam S1 = 3'd1,  
            S2 = 3'd2,  
            S3 = 3'd3,  
            S4 = 3'd4,  
            S5 = 3'd5,  
            S6 = 3'd6;
```

7.3.2 Timing Parameters

The controller uses the following timing parameters (in clock cycles) to control the duration of each state:

```
// Timing constants (in clock ticks)  
localparam T_S1 = 29, // Duration for state S1  
            T_S2 = 4,  // Duration for state S2  
            T_S3 = 19, // Duration for state S3  
            T_S4 = 4,  // Duration for state S4  
            T_S5 = 29, // Duration for state S5  
            T_S6 = 4,  // Duration for state S6  
            T_EM = 9;  // Duration for emergency mode
```

7.4 State Transition Logic

The controller follows a fixed sequence of states during normal operation:

```
// Next-state logic
always @(*) begin
    case (current_state)
        S1: next_state = S2;
        S2: next_state = S3;
        S3: next_state = S4;
        S4: next_state = S5;
        S5: next_state = S6;
        S6: next_state = S1;
        default: next_state = S1;
    endcase
end
```

7.5 Normal Operation

During normal operation, the controller cycles through states S1-S6, with each state having specific signal configurations:

- **State S1 (30 seconds):** T1 is Green, T2 is Red, pedestrians can cross at T2
- **State S2 (5 seconds):** T1 is Yellow, T2 is Red, pedestrians can cross at T2
- **State S3 (20 seconds):** T1 is Red, T2 is Red, pedestrians can cross at both T1 and T2
- **State S4 (5 seconds):** T1 is Red, T2 is Red, pedestrians can cross at both T1 and T2, buzzer2 active
- **State S5 (30 seconds):** T1 is Red, T2 is Green, pedestrians can cross at T1
- **State S6 (5 seconds):** T1 is Red, T2 is Yellow, pedestrians can cross at T1, buzzer1 active

7.6 Emergency Vehicle Priority

The controller implements priority handling for emergency vehicles with the following features:

- Emergency signals from right have higher priority than those from left
- When an emergency signal is detected, normal operation is suspended
- Emergency mode lasts for 10 seconds and can be extended if the signal persists
- During right emergency: both T1 and T2 are Red, no pedestrian crossing allowed
- During left emergency: T1 is Red, T2 is Green, no pedestrian crossing allowed

7.7 Implementation Details

7.7.1 State and Timer Registers

The controller uses several registers to track its operation:

```
reg [2:0] current_state, next_state;
reg [6:0] timer, emergency_timer;
reg emergency_right_active, emergency_left_active;
```

7.7.2 Reset Behavior

On reset, the controller initializes to a known state:

```
if (reset) begin
    current_state    <= S1;
    timer            <= 0;
    emergency_timer  <= 0;
    emergency_right_active <= 0;
    emergency_left_active <= 0;
    T1               <= GREEN;
    T2               <= RED;
    T1_WALK          <= 1'b0;
    T2_WALK          <= 1'b0;
    buzzer1          <= 1'b0;
    buzzer2          <= 1'b0;
end else begin
```

7.7.3 Emergency Logic

Emergency Right is prioritised over Emergency left, An emergency signal will suspend normal operations for 10 seconds or more depending on the duration of Emergency Signal

```
case (emergency_right_active | emergency_left_active | Emergency_Left |
    Emergency_Right)
    1'b1: begin
        // first, check Right (priority)
        case (emergency_right_active | Emergency_Right)
            1'b1: begin
                // Set active flag when emergency signal is detected
                case (Emergency_Right)
                    1'b1: emergency_right_active <= 1'b1;
                    default: begin end
                endcase

                // Check if emergency should continue
                case (emergency_timer < T_EM)
                    1'b1: emergency_timer <= emergency_timer + 1;
                    default: begin
                        // Timer hit 10 seconds, check emergency signal again
                        case (Emergency_Right)
                            1'b1: emergency_timer <= 0; // restart timer
                            default: begin
                                emergency_timer <= 0;
                                emergency_right_active <= 0;
                            end
                        endcase
                    end
                endcase

                // Set outputs for right emergency
                T1 <= RED;
                T2 <= RED;
                T1_WALK <= 1'b0;
                T2_WALK <= 1'b0;
                buzzer1 <= 1'b0;
                buzzer2 <= 1'b0;
            end

        default: begin
            // then, if not Right, check Left
            case (emergency_left_active | Emergency_Left)
                1'b1: begin
                    // Set active flag when emergency signal is detected
```

```

        case (Emergency_Left)
            1'b1: emergency_left_active <= 1'b1;
            default: begin end
        endcase

        // Check if emergency should continue
        case (emergency_timer < T_EM)
            1'b1: emergency_timer <= emergency_timer + 1;
            default: begin
                // Timer hit 10 seconds, check emergency signal
                again
                case (Emergency_Left)
                    1'b1: emergency_timer <= 0; // restart timer
                    default: begin
                        emergency_timer <= 0;
                        emergency_left_active <= 0;
                    end
                endcase
            end
        endcase

        // Set outputs for left emergency
        T1          <= RED;
        T2          <= GREEN;
        T1_WALK     <= 1'b0;
        T2_WALK     <= 1'b0;
        buzzer1     <= 1'b0;
        buzzer2     <= 1'b0;
    end
    default: begin end
endcase
end
endcase
end

```

Listing 1: Complete Traffic Light Controller Implementation

7.8 Operational Flow

The main operational flow follows this sequence:

1. Check for emergency signals (right priority over left)
2. If emergency is active, handle according to direction
3. If no emergency, proceed with normal state transitions
4. Update outputs based on current state
5. Increment appropriate timers
6. Transition to next state when timer expires

Pedestrian signals and buzzers are single-bit signals (1 for active, 0 for inactive).

8 Testbench Design

The testbench is designed to verify all aspects of the traffic light controller functionality:

8.1 Testbench Structure

```
module tb_traffic_light_controller;
    // Clock, reset and emergency signals
    reg clk, reset;
    reg Emergency_Left, Emergency_Right;

    // Output signals
    wire [1:0] T1, T2;
    wire T1_WALK, T2_WALK, buzzer1, buzzer2;

    // Module instantiation
    traffic_light_controller uut (
        .clk(clk), .reset(reset),
        .Emergency_Left(Emergency_Left),
        .Emergency_Right(Emergency_Right),
        .T1(T1), .T2(T2),
        .T1_WALK(T1_WALK), .T2_WALK(T2_WALK),
        .buzzer1(buzzer1), .buzzer2(buzzer2)
    );
```

8.2 Detailed Testbench Code Explanation

The testbench implements a comprehensive verification environment for the traffic light controller. Below is a detailed explanation of the key components:

```
'timescale 1s/1ms

module tb_traffic_light_controller;

    reg clk, reset;
    reg Emergency_Left, Emergency_Right;
    wire [1:0] T1, T2;
    wire T1_WALK, T2_WALK, buzzer1, buzzer2;

    // Instantiate your module
    traffic_light_controller uut (
        .clk(clk),
        .reset(reset),
        .Emergency_Left(Emergency_Left),
        .Emergency_Right(Emergency_Right),
        .T1(T1),
        .T2(T2),
        .T1_WALK(T1_WALK),
        .T2_WALK(T2_WALK),
        .buzzer1(buzzer1),
        .buzzer2(buzzer2)
    );
```

8.2.1 Timescale and Signal Declaration

The testbench begins with a timescale directive of 1s/1ms, meaning the time unit is 1 second with a precision of 1 millisecond. This matches the expected timing of the traffic light controller. The input signals (clk, reset, Emergency_Left, Emergency_Right) are declared as reg type, while output signals are declared as wire type.

```
// Clock generation
initial begin
    clk = 0;
    forever #0.5 clk = ~clk;
end
```

8.2.2 Clock Generation

The clock is generated with a period of 1 second (0.5s high, 0.5s low). This initial block runs continuously throughout the simulation, toggling the clock signal every 0.5 seconds.

```
// Light name string variables (for display)
reg [8*6:1] T1_str, T2_str;

always @* begin
    case (T1)
        2'b00: T1_str = "GREEN ";
        2'b01: T1_str = "YELLOW";
        2'b10: T1_str = "RED   ";
        default: T1_str = "UNKWN ";
    endcase

    case (T2)
        2'b00: T2_str = "GREEN ";
        2'b01: T2_str = "YELLOW";
        2'b10: T2_str = "RED   ";
        default: T2_str = "UNKWN ";
    endcase
end
```

8.2.3 Signal Interpretation

This section defines string variables to convert the binary traffic light signals into human-readable text. The always block continuously updates these strings based on the current state of T1 and T2 signals. This makes the simulation output more readable and easier to interpret.

```
// Print when anything changes
initial begin
    $monitor("Time: %0d | T1: %s | T2: %s | T1_WALK = %b | T2_WALK = %b |
        Buzzer1 = %b | Buzzer2 = %b | E_Left = %b | E_Right = %b",
        $time, T1_str, T2_str, T1_WALK, T2_WALK, buzzer1, buzzer2,
        Emergency_Left, Emergency_Right);
end
```

8.2.4 Monitoring and Reporting

The \$monitor statement automatically prints a formatted message whenever any of the monitored signals change. This provides a continuous log of the system state throughout the simulation, including the time, traffic light states, pedestrian signals, buzzers, and emergency inputs.

```
// Stimulus
integer i;
initial begin
    // Start VCD file generation
    $dumpfile("traffic_light_controller.vcd");
    $dumpvars(0, tb_traffic_light_controller);

    $display("\n== T-Junction Traffic Light Controller Simulation Started
        ==\n");

    Emergency_Left = 0;
    Emergency_Right = 0;
    reset = 1;
    #1;
    reset = 0;
    $display("== Reset at t=%0d ==", $time);
end
```

8.2.5 Simulation Setup and Reset

The main stimulus block begins by setting up VCD (Value Change Dump) file generation for waveform viewing. It then initializes all input signals and applies a reset pulse for 1 second. The \$display statements provide clear markers in the simulation output to indicate the beginning of each test phase.

```
// Normal operation - let it go through a complete cycle
$display("\n=== Normal Operation - Full Cycle ===");
#100;

// Test Case 1: Short Emergency Left (< T_EM)
$display("\n=== Test Case 1: Short Emergency Left (<T_EM) ===");
Emergency_Left = 1;
#2;
Emergency_Left = 0;
// Wait for emergency to complete (should be T_EM = 9 seconds)
#15;
```

8.2.6 Test Case Implementation

Each test case follows a similar pattern:

1. A display message indicating the start of the test case
2. Setting the appropriate emergency signals
3. Waiting for specific periods
4. Clearing the emergency signals
5. Waiting for the system to return to normal operation

The first test case verifies that a short emergency signal (2 seconds) will still maintain the emergency state for the minimum duration ($T_{EM} = 9$ seconds).

```
// Test Case 2: Long Emergency Left (> T_EM)
$display("\n=== Test Case 2: Long Emergency Left (>T_EM) ===");
Emergency_Left = 1;
#15;
Emergency_Left = 0;
// Wait for emergency to complete
#15;
```

Test Case 2 verifies that a long emergency signal (15 seconds) will maintain the emergency state for the full duration of the signal, which is longer than the minimum T_{EM} .

```
// Test Case 5: Priority test - Both emergencies active, Right should
take priority
$display("\n=== Test Case 5: Priority Test - Both Emergencies Active
===");
Emergency_Left = 1;
Emergency_Right = 1;
#15;
Emergency_Right = 0;
#5;
Emergency_Left = 0;
// Wait for emergency to complete
#15;
```

Test Case 5 verifies the priority mechanism by activating both emergency signals simultaneously. The Right emergency should take precedence over the Left emergency.

```
// Test Case 6: Overlapping emergencies - Left starts, then Right
interrupts
$display("\n=== Test Case 6: Overlapping Emergencies - Left then Right
===");
```

```

Emergency_Left = 1;
#5;
Emergency_Right = 1;
#5;
Emergency_Left = 0;
#5;
Emergency_Right = 0;
// Wait for emergency to complete
#15;

```

Test Cases 6 and 7 verify the handling of overlapping emergency signals, testing both scenarios where Left starts first and where Right starts first.

```

// Test Case 8: Back-to-back emergencies
$display("\n== Test Case 8: Back-to-back Emergencies ==");
Emergency_Right = 1;
#5;
Emergency_Right = 0;
#12; // Just after first emergency ends
Emergency_Left = 1;
#5;
Emergency_Left = 0;
// Wait for emergency to complete
#15;

// Final normal operation
$display("\n== Final Normal Operation ==");
#50;

$display("\n== Simulation Complete ==\n");
$finish;
end
endmodule

```

8.2.7 Sequential Test Cases and Simulation End

The final test cases verify back-to-back emergency handling and the return to normal operation. The simulation concludes with 50 seconds of normal operation to verify that the controller returns to its standard cycle. The \$finish statement terminates the simulation.

8.3 Test Scenarios

The testbench implements eight comprehensive test cases:

1. Normal operation - Full cycle
2. Short Emergency Left ($< T_{EM}$)
3. Long Emergency Left ($> T_{EM}$)
4. Short Emergency Right ($< T_{EM}$)
5. Long Emergency Right ($> T_{EM}$)
6. Priority test - Both emergencies active
7. Overlapping emergencies - Left then Right
8. Overlapping emergencies - Right then Left
9. Back-to-back emergencies

8.4 Simulation Features

- Clock generation with 1s period
- Signal monitoring with formatted output
- VCD file generation for waveform analysis
- Descriptive test case labeling

9 Waveform Analysis

9.1 Simulation Results

The waveform captures show the behavior of the traffic light controller under various test scenarios:

9.2 Output in Terminal

```
VCD info: dumpfile traffic_light_controller.vcd opened for output.

=== T-Junction Traffic Light Controller Simulation Started ===

Time: 0 | T1: GREEN | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = x | E_Left =
0 | E_Right = 0
=== Reset at t=1 ===

=== Normal Operation - Full Cycle ===
Time: 2 | T1: GREEN | T2: RED | T1_WALK = 0 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left =
0 | E_Right = 0
Time: 32 | T1: YELLOW | T2: RED | T1_WALK = 0 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left =
0 | E_Right = 0
Time: 37 | T1: RED | T2: RED | T1_WALK = 1 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left =
0 | E_Right = 0
Time: 57 | T1: RED | T2: RED | T1_WALK = 1 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 1 | E_Left =
0 | E_Right = 0
Time: 62 | T1: RED | T2: GREEN | T1_WALK = 1 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left =
0 | E_Right = 0
Time: 92 | T1: RED | T2: YELLOW | T1_WALK = 1 | T2_WALK = 0 | Buzzer1 = 1 | Buzzer2 = 0 | E_Left =
0 | E_Right = 0
Time: 97 | T1: GREEN | T2: RED | T1_WALK = 0 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left =
0 | E_Right = 0

=== Test Case 1: Short Emergency Left (<T_EM) ===
Time: 101 | T1: GREEN | T2: RED | T1_WALK = 0 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 1 | E_Right = 0
Time: 102 | T1: RED | T2: GREEN | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 1 | E_Right = 0
Time: 103 | T1: RED | T2: GREEN | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0
Time: 112 | T1: GREEN | T2: RED | T1_WALK = 0 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0

=== Test Case 2: Long Emergency Left (>T_EM) ===
Time: 118 | T1: GREEN | T2: RED | T1_WALK = 0 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 1 | E_Right = 0
Time: 119 | T1: RED | T2: GREEN | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 1 | E_Right = 0
Time: 133 | T1: RED | T2: GREEN | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0
Time: 139 | T1: GREEN | T2: RED | T1_WALK = 0 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0

=== Test Case 3: Short Emergency Right (<T_EM) ===
```

Current State	Duration (seconds)	Next State	Condition
S1	30	S2	Timer expires
S2	5	S3	Timer expires
S3	20	S4	Timer expires
S4	5	S5	Timer expires
S5	30	S6	Timer expires
S6	5	S1	Timer expires
Emergency	10	Previous state	Emergency ends

Table 2: State Transition Table

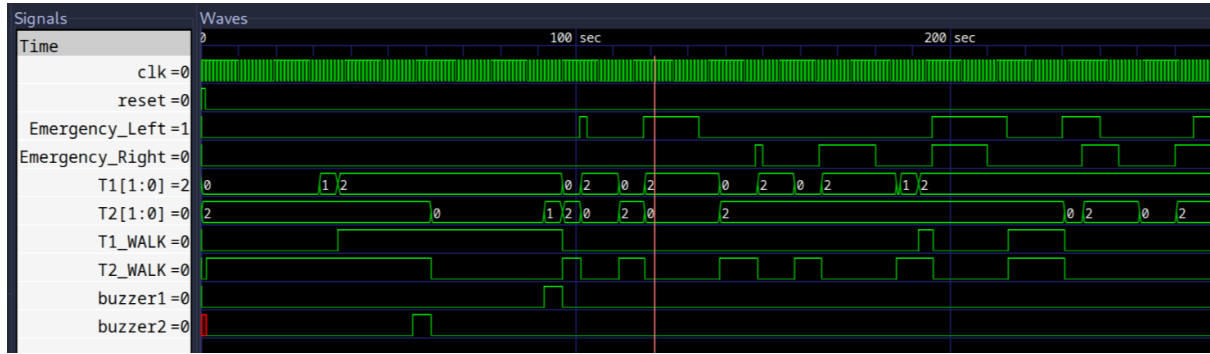


Figure 3: Waveform showing first half of simulation (0-200s)

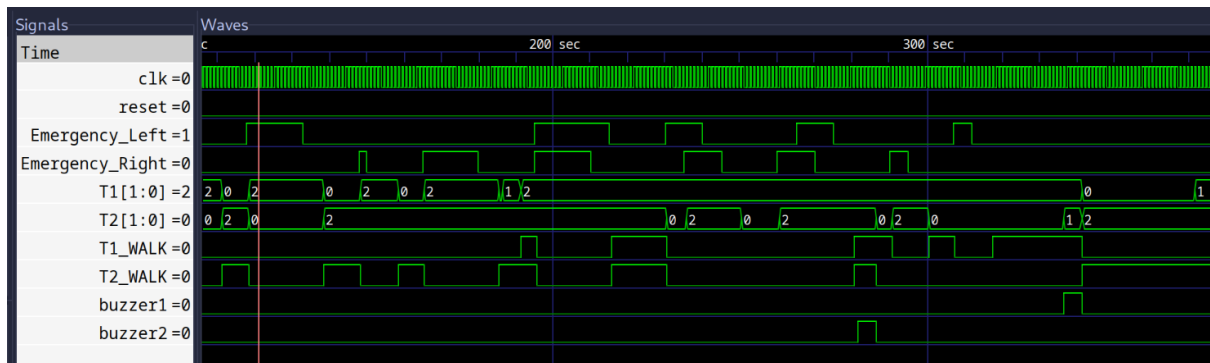


Figure 4: Waveform showing second half of simulation (200-300s)

```

Time: 148 | T1: GREEN | T2: RED | T1_WALK = 0 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 1
Time: 149 | T1: RED | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 1
Time: 150 | T1: RED | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0
Time: 159 | T1: GREEN | T2: RED | T1_WALK = 0 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0

=== Test Case 4: Long Emergency Right (>T_EM) ===
Time: 165 | T1: GREEN | T2: RED | T1_WALK = 0 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 1
Time: 166 | T1: RED | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 1
Time: 180 | T1: RED | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0
Time: 186 | T1: GREEN | T2: RED | T1_WALK = 0 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0
Time: 187 | T1: YELLOW | T2: RED | T1_WALK = 0 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0
Time: 192 | T1: RED | T2: RED | T1_WALK = 1 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0

=== Test Case 5: Priority Test - Both Emergencies Active ===
Time: 195 | T1: RED | T2: RED | T1_WALK = 1 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 1 | E_Right = 1
Time: 196 | T1: RED | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 1 | E_Right = 1
Time: 210 | T1: RED | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 1 | E_Right = 0
Time: 215 | T1: RED | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0
Time: 216 | T1: RED | T2: RED | T1_WALK = 1 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0

=== Test Case 6: Overlapping Emergencies - Left then Right ===
Time: 230 | T1: RED | T2: RED | T1_WALK = 1 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 1 | E_Right = 0
Time: 231 | T1: RED | T2: GREEN | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 1 | E_Right = 0
Time: 235 | T1: RED | T2: GREEN | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 1 | E_Right = 1
Time: 236 | T1: RED | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 1 | E_Right = 1
Time: 240 | T1: RED | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 1
Time: 245 | T1: RED | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0
Time: 251 | T1: RED | T2: GREEN | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0

=== Test Case 7: Overlapping Emergencies - Right then Left ===
Time: 260 | T1: RED | T2: GREEN | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 1
Time: 261 | T1: RED | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 1
Time: 265 | T1: RED | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 1 | E_Right = 1
Time: 270 | T1: RED | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 1 | E_Right = 0
Time: 275 | T1: RED | T2: RED | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
= 0 | E_Right = 0

```

```

Time: 281 | T1: RED      | T2: RED      | T1_WALK = 1 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
          = 0 | E_Right = 0
Time: 282 | T1: RED      | T2: RED      | T1_WALK = 1 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 1 | E_Left
          = 0 | E_Right = 0
Time: 287 | T1: RED      | T2: GREEN    | T1_WALK = 1 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
          = 0 | E_Right = 0

=== Test Case 8: Back-to-back Emergencies ===
Time: 290 | T1: RED      | T2: GREEN    | T1_WALK = 1 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
          = 0 | E_Right = 1
Time: 291 | T1: RED      | T2: RED      | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
          = 0 | E_Right = 1
Time: 295 | T1: RED      | T2: RED      | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
          = 0 | E_Right = 0
Time: 301 | T1: RED      | T2: GREEN    | T1_WALK = 1 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
          = 0 | E_Right = 0
Time: 307 | T1: RED      | T2: GREEN    | T1_WALK = 1 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
          = 1 | E_Right = 0
Time: 308 | T1: RED      | T2: GREEN    | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
          = 1 | E_Right = 0
Time: 312 | T1: RED      | T2: GREEN    | T1_WALK = 0 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
          = 0 | E_Right = 0
Time: 318 | T1: RED      | T2: GREEN    | T1_WALK = 1 | T2_WALK = 0 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
          = 0 | E_Right = 0

=== Final Normal Operation ===
Time: 337 | T1: RED      | T2: YELLOW   | T1_WALK = 1 | T2_WALK = 0 | Buzzer1 = 1 | Buzzer2 = 0 | E_Left
          = 0 | E_Right = 0
Time: 342 | T1: GREEN    | T2: RED      | T1_WALK = 0 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
          = 0 | E_Right = 0
Time: 372 | T1: YELLOW   | T2: RED      | T1_WALK = 0 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
          = 0 | E_Right = 0
Time: 377 | T1: RED      | T2: RED      | T1_WALK = 1 | T2_WALK = 1 | Buzzer1 = 0 | Buzzer2 = 0 | E_Left
          = 0 | E_Right = 0

=== Simulation Complete ===

main_tb.v:159: $finish called at 377000 (1ms)

```

9.3 Key Observations

9.3.1 Normal Operation

In normal operation (0-70s), the controller cycles through states S1-S6:

- T1 transitions: Green → Yellow → Red
- T2 transitions: Red → Red → Green → Yellow
- Walk signals alternate with appropriate timing
- Buzzers activate during pedestrian crossing periods

9.3.2 Emergency Response

The waveforms demonstrate correct emergency behavior:

- At 70s: Short Emergency_Left activates for 2s but maintains emergency state for full T.EM duration
- At 90s: Long Emergency_Left activates and maintains state for the full duration of the signal
- At 130s: Emergency_Right takes priority when both signals are active
- At 160-180s: Overlapping emergencies are handled correctly with Right having priority